

## Project Progress Report - Day 2 (11/06/2026)



### Overview

On Day 2, we successfully transitioned our local environment into a full robotics testing suite by tackling hardware virtualization, operating system level installations, and package conflict resolutions. In parallel, we dove into the core algorithmic architecture of the project, focusing on the Slime Mold Algorithm (SMA), biological stigmergy, and alternative swarm optimization paths.



### Infrastructure & Environment Setup

#### 1. Hardware & System Configuration

- BIOS Hardware Virtualization: Accessed the Lenovo AMD firmware settings and explicitly enabled AMD V(TM) Technology (SVM Mode) to unlock underlying CPU virtualization constraints.
- Windows Subsystem for Linux (WSL): Deployed a clean instance of Ubuntu 24.04 / 26.04 using the Windows Administrator Terminal to act as our primary Linux development environment.

#### 2. Simulator Deployment (Gazebo)

- The Problem: Direct Ubuntu package upgrades (apt) hit structural dependency locks because the newer system environment ran Python 3.13, which conflicted with standard downstream binary builds.
- The Fix: Leveraged Pixi (a modern, isolated package environment manager) to bypass OS-level system locks.
- Verification: Successfully fetched and ran the simulation engine interface directly from the terminal via:

```
gazebo worlds/shapes.world
```

The engine initialized cleanly with hardware graphics acceleration, rendering a steady 3D grid layout complete with physical objects (cylinder, cube, and sphere nodes).

### Algorithmic Framework & Research

We initiated the conceptual design phase for the system's control layers, focusing heavily on swarm intelligence.

#### 1. Slime Mold Algorithm (SMA) & Stigmergy

- Bio-Inspiration: Evaluated the behavioral dynamics of *Physarum polycephalum*. SMA models how simple agents form optimal path networks between food sources based on dynamic feedback.
- Stigmergy Mechanisms: Explored how agents communicate indirectly through environmental modifications (e.g., slime trails). In computational robotics, this translates to updating a global or decentralized cost-map matrix that guides agent exploration without requiring direct peer-to-peer radio messaging.

## 2. Comparative Algorithmic Candidates for the Project

To find the absolute best fit for our multi-agent/robotics routing, we analyzed and cross-compared a collection of alternative optimization frameworks:

| Algorithm                                | Core Strength                                                          | Project Relevance / Application                                                                      |
|------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Slime Mold Algorithm (SMA)</b>        | Exceptional at adaptive path planning and dynamic network routing.     | Primary Candidate: Ideal for routing multi-robot fleets through changing or unknown obstacle fields. |
| <b>Ant Colony Optimization (ACO)</b>     | Highly efficient at discrete combinatorial problem spaces.             | Alternative: Can be combined with SMA to optimize discrete waypoint sequences.                       |
| <b>Particle Swarm Optimization (PSO)</b> | Rapid convergence profiles in continuous, high-dimensional spaces.     | Alternative: Useful for tuning control loop parameters or locating absolute coordinate gradients.    |
| <b>Artificial Bee Colony (ABC)</b>       | Excellent balance between deep exploration and exploitation behaviors. | Alternative: Well suited for distributing task-allocation roles across different robot clusters.     |